The Dimensions of SHA

Randy Merritt

Christmas, 2025

Let's Start At The Beginning

What is a Hash?

A **hash** is the output of a **hash function** — a mathematical algorithm that takes input data of any size (like a password, file, or message) and converts it into a fixed-length string of characters (usually numbers and letters), which appears random. A general hash function (also called a non-cryptographic hash function) is designed for speed and efficiency in non-adversarial contexts.

Primary goal:

Quickly map data of arbitrary size to a fixed-size value (hash) for tasks like:

- Hash tables (dictionaries, sets)
- Bloom filters
- Checksums for error detection (e.g., CRC32)
- Cache keys
- Database indexing

Key properties:

- Fast computation (optimized for speed)
- Good distribution (minimizes collisions for typical data)
- Deterministic (same input → same output)
- May have weaknesses like predictable collisions, susceptibility to deliberate attacks

These hashes are not designed to withstand malicious attempts to find collisions or reverse the hash.

SHA

SHA. The Secure Hash Algorithm. Developed by the U.S. National Security Agency (NSA) this family of cryptographic hash functions is designed to withstand malicious attempts to find collisions or reverse the hash. It's widely used for data integrity verification, digital signatures, and cybersecurity applications. The algorithm's ability to create a unique, irreversible digital representation of data makes it crucial in modern digital security infrastructure.

The Secure Hash Algorithm (SHA) is significant for several reasons:

1. **Password Storage**
   Websites store hashes (not plain passwords). When you log in, they hash your input and compare it to the stored hash.

2. **Data Integrity**
   Verify a downloaded file's hash matches the publisher's provided hash to ensure it wasn't corrupted or tampered with.

3. **Digital Signatures & Certificates**
   Used in public-key infrastructure.

4. **Blockchain & Cryptocurrencies**
   Bitcoin uses SHA-256 to link blocks.

5. **Hash Tables (Data Structure)**
   Used for fast data lookup (key → hash → storage index).

A Simple Digital Signature

Let's talk digital signatures. Why? A couple reasons: 1) In our current age we need a way of indisputable authentication of a digital work. When you publicize something on a public network and you want proper attribution, you want an uncompromisable method of signing your work; an uncrackable digital signature. 2) With the advent of quantum computers the race is now on between the code breakers that want to break your protection of authentication and those who need a method to assure that the assigned signature is not modified.

We focus this discussion on digital signatures, particularly signatures that can be done solely using the SHA. More secure signatures involve certificates and authentication methods, but here we explore signatures that can be hardened using only the SHA.

To apply a digital signature you need two components: a key and it's corresponding lock. The key is a character string only you know: there is no need to share it. Indeed, as we will show, the key need not be some random arcane series of characters. It can be a simple easy-to-remember string, where you may not need to even write it down.

The corresponding lock is the resultant string of characters generated by a selected version of SHA using the key as input; it is the SHA that intimately binds the key with it's lock. The key is what you keep secret; the lock you make public however you wish.

And this is where the SHA makes its contribution. When one publishes a work, you publish your lock string along with it. This is your secure digital signature. Your key you do not publish -- it's your secret. Should a lawsuit occur where there is a dispute over who's the original author, you have only to show the court that you, and only you, have the key that matches the lock. A thief would never be able to do that.

The service used for the examples in this paper is *hashingservice.com*. It shows (currently) 53 hash functions to choose to encrypt a string, and (currently) 11 versions of the SHA.

These examples use SHA256.

key: cat
lock:
77af778b51abd4a3c51c5ddd97204a9c3ae614ebccb75a606c3b6865aed6744e

key: How Now Brown Cow
lock:
3ab0cc0c76572bb7dc792b6fa4050a1fda3cf62707a76c3bf63c60053b3189a1

key: How Now Brow∏ Cow (Apple keyboard shift-option-P = ∏)
lock:
30e260fc54ec85b40d0844e7757819ef6af824bd60f8c8550b355001f5686894

key: Four score and seven years ago our fathers brought forth on this continent
lock:
bb9a21c8d4fcf3354d1d8c81d6a4a0ab9c7a8443a43c6e7cbd03cb604eb42348

Note this entire PDF paper was signed by a SHA256 hash on an Apple computer using "shasum -a 256 ./TheDimensionsOfSHA.pdf". It's digital signature is in the accompanying README file.

The 8 Dimensions of SHA

The design of the SHA is a remarkable achievement in that these largely separate characteristics, or dimensions, combine to thwart any attempt to reduce the search for a key by inference on the public lock (hash). Brute-force guesses are the only option.

1. Collision-resistance. A prime achievement of the Secure Hash Algorithm is the absolute uniqueness between an input string (key) and it's computed output string (lock). Pass the SHA a string of characters and the resultant output string belongs to that input string and *only* that input string; no other input can generate that output. A corollary to this characteristic is that a given input will *always* produce the same output.

2. Asymmetry. The SHA (as with most hashes) is purely asymmetric; the transform of plain text to a hash is *strictly* one-way. This one-way only string encryption makes it's easy to derive a lock from a key but impossible to derive a key from a lock. This dimension alone forces the brute-force approach.

3. Determinate. SHA265 generates a 65-character result -- *always*. It matters not the length of the key string. This promotes further security in that there's no way to reduce a key search by inferring it's possible length from a lock string since SHA results are always the same length.

4. The Avalanche Affect. With SHA *any* change in a key string not only yields a different lock string, but a *completely* different lock string. This eliminates any possibility of using a converging sequence of lock strings to narrow a search for the corresponding key.

5. Range. This hardens the SHA. One is not limited to what characters can be used in an input string, because the SHA works on bits. It ignores the type of data passed as it's input. *Any* combination of letters, numbers, punctuation characters, Tabs and Space. Indeed, *anything* can be used for a key: an image, a video, spreadsheet, text, PDF — anything. So instead of thinking string, think algorithmic. Pick some arcane, obscure file and hash that — that's all you have to remember for your key.

6. Variants. As noted, hash service website *https://hashingservice.com/* (currently) offers 11 separate SHA variations that can be used. Over time, such selections may increase. So any crack attempt must include invoking *all* variations of SHA in it's hunt for the key.

7. The Cascade Affect. This further hardens the SHA. It introduces an ambiguity even a quantum computer cannot get past. With SHA you can nest hashes.

To wit, using SHA256:

key: cat
lock:
77af778b51abd4a3c51c5ddd97204a9c3ae614ebccb75a606c3b6865aed6744e

key:
77af778b51abd4a3c51c5ddd97204a9c3ae614ebccb75a606c3b6865aed6744e
lock:
436b24bc69a5806a6cb84ece619826ffcb7db152164647df954cfb10087ed4cc

key:
436b24bc69a5806a6cb84ece619826ffcb7db152164647df954cfb10087ed4cc
lock:
be0bd5f59e3c8336ad10b03525f504cafb4ba73d1efdb93ce874c0c990b2ea64

. . .

This key/lock combination becomes — key = cat, lock =
be0bd5f59e3c8336ad10b03525f504cafb4ba73d1efdb93ce874c0c990b2ea64.

This example chose 3 nest levels. The ambiguity is the ". . ." One can write a simple Python program to cascade SHA iterations to *any* level. A crack attempt must then confront the issue "*could this be a nested hash and how deep does the nest go?*" (which you also keep secret).

8. Obsolescence. All information goes stale. No longer useful. No longer needed. Secret classifications now moot. As our capacity to process, disseminate, and communicate information accelerates, even the deepest secrets don't last long.

Example:
The USSR's Tsar Bomba. Detonated on 30 October 1961, the most powerful thermonuclear bomb ever built. Yet, now, in the last days of 2025, a scant 64 years later, the actual design and construction of such a weapon has not only lost it's deep classification, there are detailed animation videos on YouTube that detail its actual design and operation that yielded the largest thermonuclear detonation on record.
https://www.youtube.com/watch?v=hvdW156cTk4

What? Are you kidding? How can that be? Simple. It's no secret — everybody knows how to build it.

While obsolescence is not strictly a direct characteristic of the SHA, it is tightly tied to it. What is arguably the key characteristic of the SHA is the sum of its dimensions make it so computationally daunting, so arduous that the key decision to attempt to crack a SHA hash is "*can the crack be accomplished before the effort yields a result that's useless?*". With nothing more than a string of random characters to go on (the lock string), one has to think carefully before dedicating significant resources to an attempt that carries a strong probability that it will come to nothing.

So any attempt to crack a key/lock combination must take into account the time dimension. The longer the time needed to find a key, the shorter the time remains before the value of the underlying information expires: if information value has an expiration date, so does a SHA lock.

Appendix I — An Example

The Character Domain.

The keyboard on an Apple Mac can generate four unique characters per key. There are 26 alphabet keys, 10 number keys, and 11 punctuation keys. For each if these keys you can: tap the key, hold the Shift key and tap, hold the Shift + Option key and tap, or hold just the Option key and tap. Each of these key combinations yields a distinct character.

Plus, there is the Tab key and the Space bar — that's two more.

This totals (26+10+11) * 4 + 2 = **190** characters to choose for a key string.

Start with an easy-to-remember, nonsensical sentence, and then modify it to include some easy-to-remember set of numbers or special characters (non-alpha, non-numeric). We advocate a nonsensical phrase or sentence to thwart a crack strategy that would start with phrases, quotes, names, places, etc.

My example: How Now Brown Cow sans a mosquito

Base key string: How Now Brown Cow sans a mosquito
Enhanced key string: How Now Brown Cow an mosquito
(is shift-option-K on a Mac)

Key: How Now Brown Cow an mosquito
SHA256 lock:
c6dc3620aec8a88d15dcbb53d1c5ac8379d0997445d47b6da2d4b4d15593622c

[Note: Nowhere in any natural language lexicon will you find the phrase "How Now Brown Cow sans a mosquito", much less "How Now Brown Cow an mosquito". But it's not that hard to remember. This is one of several such nonsensical phrases from the author, this one now sacrificed for the purpose of illustration.]

How Now Brown Cow an mosquito = 36 characters.

So,

How many 36-character combinations can be derived from 190 keyboard character choices, where characters can be repeated and order does not matter?

In set combinatorics the formula reduces to "two hundred twenty-five choose thirty-six", which yields ~10^{43} combinations.

or, precisely,

664,475,247,610,481,977,808,376,880,047,081,365,034,800

That's — *six hundred sixty-four duodecillion, four hundred seventy-five undecillion, two hundred forty-seven decillion, six hundred ten nonillion, four hundred eighty-one octillion, nine hundred seventy-seven septillion, eight hundred eight sextillion, three hundred seventy-six quintillion, eight hundred eighty quadrillion, forty-seven trillion, eighty-one billion, three hundred sixty-five million, thirty-four thousand, eight hundred* — guesses.

The code breakers have met their match. No supercomputer devised has anywhere near the capability to process half this many SHA attempts in any length of time before the underlying information the SHA was meant to protect has become either obsolete or has been made public. And this assumes a level 1 hash: this doesn't begin to account for the cascade issue.

The final hope is the quantum computer.

Appendix II — A Note on Quantum Computers

The issue with computing at the quantum level is noise. Not just electrical noise but also spatial noise. Particularly, noise from outer space. If a quantum computer is truly delving into computation at the quantum level, noise will be a significant factor to it's reliability. And a big contributor in that reliability? The neutrino. Why?

A slight departure. Super-Kamiokande. A Nobel prize winning neutrino detector. How does this matter?

"*Although each neutrino is extremely small, neutrinos are the second most abundant particles in the universe only after light. Many neutrinos are pouring down on the earth from the sun and stars. Neutrinos are also produced by the collision between radiation from space (cosmic rays) and the earth's atmosphere. (Even bananas emit neutrinos!) Since a neutrino is so small and does not feel the electromagnetic force, it easily goes through even the earth. Therefore, neutrinos not only pour from the sky, but also come from the backside of the earth and go into the sky. Hundreds of trillions of neutrinos from the sun are passing through our bodies every second.*" — https://www-sk.icrr.u-tokyo.ac.jp/en/sk/about/5min/

"*<u>Hundreds of trillions</u> of neutrinos from the sun are passing through our bodies <u>every second</u>.*" But, we also need to include every star in the Milky Way plus every galaxy that we can see with our telescopes. Because if we can see those stars and galaxies, we can "see" their neutrinos.

And there's the rub. While the neutrino is extraordinarily small, that's offset by the sheer magnitude of the continuous flow of these particles. Note neutrinos travel at light speed no matter what medium (stars, planets, moons, ...) that they encounter.

"*Super-Kamiokande works by recording collisions of neutrinos in a 50,000 ton water tank. It is located one kilometer underground to eliminate interference from other particles. About 30 neutrinos are observed per day.*"

30 neutrinos per day doesn't sound like much, but this value can also fluctuate. New supernovas are detected via an increase in the neutrino count in these detectors.

But, here's the essential question:

What does a qubit do when it's hit by a neutrino?

The neutrino connection to quantum computing is reliability and any chance to break encryption hashes in any practical timeframe will require nothing short of a quantum computer. How long do you tie up a quantum computer grinding through an enormous series of hash calculations before it faults from a neutrino interaction? And how does that computer recover from the interaction? If it can't "pick up where it left off" then it has to restart from the beginning, begging the question what's to prevent this from happening again? And again?

To summarize from the previous discussions, a SHA hash can only be found by brute force: potentially *all* possible strings of *all* lengths comprised of *all* combinations of the 190 characters of a keyboard, followed by running *each* guess through several versions of SHA, plus running *each* guess through an arbitrary level of cascading SHA encodings.

And this only addresses a obscure character string. You add to this *any* file as a key, then how does a quantum computer (heck, any computer!) find that key.

So, with nothing more to go on except the public lock (hash) string

c6dc3620aec8a88d15dcbb53d1c5ac8379d0997445d47b6da2d4b4d15593622c

have at it. Fire up those expensive quantum computers and get cracking — and good luck.

Appendix III — Python Examples

1. Basic Hashing Example

```
import hashlib

# Create a SHA-256 hash object
hash_object = hashlib.sha256()

# Update with data (can be called multiple times)
hash_object.update(b"Hello, ")
hash_object.update(b"World!")

# Get the hexadecimal digest
hex_digest = hash_object.hexdigest()
print(f"SHA-256: {hex_digest}")
```

2. All SHA Variants Available

```
import hashlib

# Available algorithms (Python 3.11+)
print("Available algorithms:", hashlib.algorithms_guaranteed)

# Typical output: {'sha3_256', 'sha256', 'sha512', 'blake2s', 'sha3_512',
#                  'md5', 'sha3_224', 'sha384', 'sha224', 'sha3_384',
#                  'blake2b', 'sha1'}

# Most commonly used:
data = b"Hello, World!"

print(f"SHA-1:      {hashlib.sha1(data).hexdigest()}")
print(f"SHA-256:  {hashlib.sha256(data).hexdigest()}")
print(f"SHA-512:  {hashlib.sha512(data).hexdigest()}")
print(f"SHA3-256:{hashlib.sha3_256(data).hexdigest()}")
print(f"SHA3-512: {hashlib.sha3_512(data).hexdigest()}")
```

3. Timing Different SHA Algorithms

```
import hashlib
import time

def benchmark(data, algorithm):

    """Benchmark hash algorithms"""
    start = time.perf_counter()

    # Hash 1000 times
    for _ in range(1000):
        hashlib.new(algorithm, data).hexdigest()
        elapsed = time.perf_counter() - start
    return elapsed

data = b"A" * 1000  # 1KB of data

algorithms = ['sha1', 'sha224', 'sha256', 'sha384', 'sha512', 'sha3_256', 'sha3_512']

print("Algorithm | Time (ms) | Digest Length”)
print("-" * 40)

for algo in algorithms:
        time_ms = benchmark(data, algo) * 1000
        length = len(hashlib.new(algo, b"test").hexdigest()) * 4  # bits
        print(f"{algo:10} {time_ms:10.2f} {length:8} bits")
```

4. The Cascade (game over)

```
import hashlib

def generate_sha256_hash(input_string):
    # Convert string to bytes (required for hashing)
       input_bytes = input_string.encode('utf-8')

    # Create SHA-256 hash object
       hash_object = hashlib.sha256()

    # Update hash object with input bytes
      hash_object.update(input_bytes)

    # Get hexadecimal representation of the hash
       hex_digest = hash_object.hexdigest()

       return hex_digest

def main():
    # choose your key (your secret)
       key = "cat"
    # choose your nesting level (also your secret)
       nest = 100
    # lock starts as key
       lock = key

    # Apple M2 MacBook Air — fraction of a second
      for i in range(1,nest):
          lock = generate_sha256_hash(lock)

    print(f"Key:  {key}")
    print(f"Lock: {lock}")
    print(f"Nest: {nest}")

    if __name__ == "__main__":main()
```